

LAB 3 : Empirical Analysis of Selection sort

Theory:

Selection Sort is a simple comparison-based algorithm that divides the input list into two parts: a sorted sublist built up from left to right at the front of the list, and a sublist of the remaining unsorted items that occupy the rest of the list. The algorithm proceeds by finding the smallest element in the unsorted sublist, swapping it with the leftmost unsorted element (putting it in sorted order), and moving the sorted boundary one element to the right.

Because it must always scan the entire remaining unsorted section to blindly figure out what the next minimum is, it remains highly inefficient. It forces the exact same number of operations regardless of whether the array is fully sorted, reverse sorted, or entirely random.

Algorithm:

SelectionSort(arr, n):

```
    FOR i from 0 to n - 2:
        min_index = i
        FOR j from i + 1 to n - 1:
            IF arr[j] < arr[min_index] THEN
                min_index = j
            END IF
        END FOR
        IF min_index != i THEN
            SWAP arr[i] AND arr[min_index]
        END IF
    END FOR
```

Time complexity of Selection sort:

- **Best Case:** $O(n^2)$ (*Always has to scan the rest of the list*)
- **Average Case:** $O(n^2)$
- **Worst Case:** $O(n^2)$

Source Code:

```
#include <stdio.h>

#include <stdlib.h>

#include <time.h>

unsigned long long count = 0;

void generate(int *arr, int n) {

    for (int i = 0; i < n; i++) { arr[i] = rand() % 1000; count++; }

}

void generateINC(int *arr, int n) {

    arr[0] = rand() % 1000; count++;

    for (int i = 1; i < n; i++) { arr[i] = arr[i - 1] + (rand() % 50 + 1); count += 2; }

}

void generateDEC(int *arr, int n) {

    arr[0] = rand() % 5000; count++;

    for (int i = 1; i < n; i++) { arr[i] = arr[i - 1] - (rand() % 50 + 1); count += 2; }

}

void selectionsort(int *arr, int n) {

    for (int i = 0; i < n - 1; i++) {

        int min = i; count++;

        for (int j = i + 1; j < n; j++) {

            count++;

            if (arr[j] < arr[min]) min = j;

        }

        if (min != i) {
```

```

        int temp = arr[i]; arr[i] = arr[min]; arr[min] = temp;

        count += 3;

    }

}

}

void print() { printf("Count : %llu\n", count); }

int main() {

    srand(time(NULL));

    for (int size = 1000; size <= 10000; size += 1000) {

        printf("\n023bscit009_binesh\nFor input size:: %d", size);

        int *a = (int *)malloc(sizeof(int) * size);

        int *b = (int *)malloc(sizeof(int) * size);

        int *c = (int *)malloc(sizeof(int) * size);


        printf("\nAscending:: \n"); count = 0;

        generateINC(a, size); selectionsort(a, size); print();

        printf("Random:: \n"); count = 0;

        generate(b, size); selectionsort(b, size); print();

        printf("Descending:: \n"); count = 0;

        generateDEC(c, size); selectionsort(c, size); print();

        free(a); free(b); free(c);

    }

    return 0;

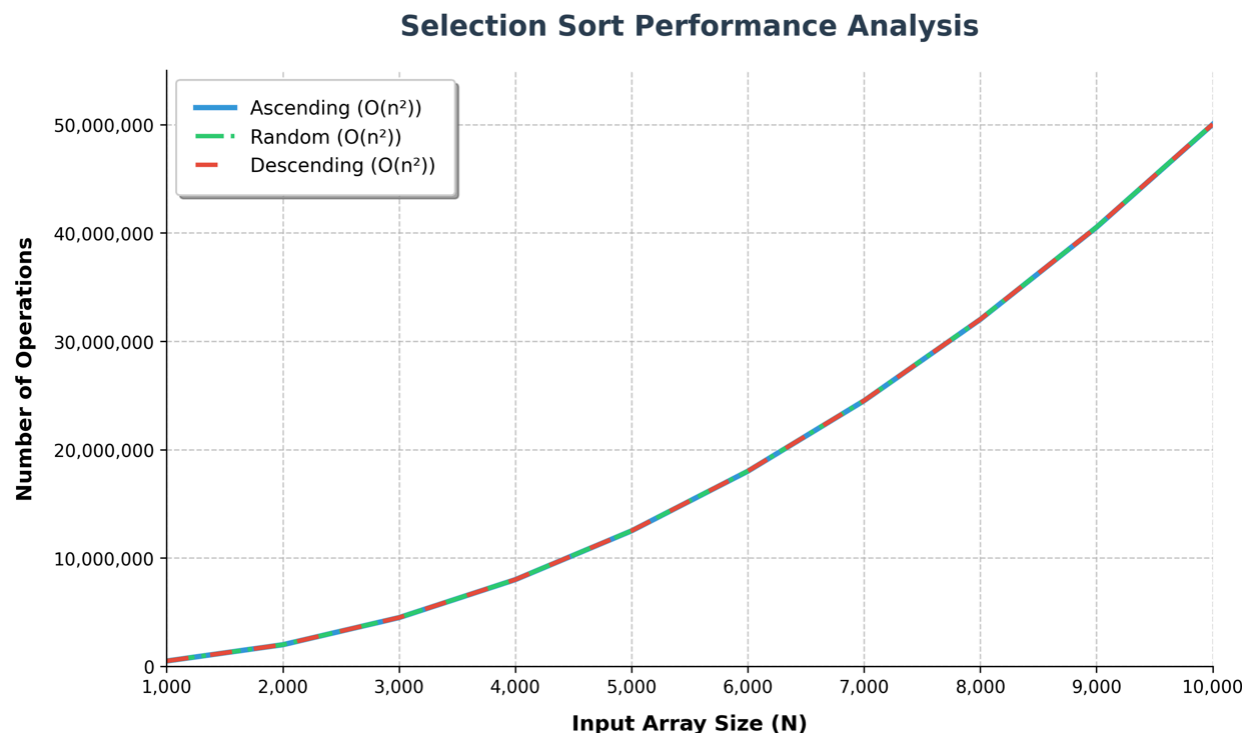
}

```

Data Table:

Input Size	Ascending	Random	Descending
1,000	502,498	504,475	503,998
2,000	2,004,998	2,008,972	2,007,998
3,000	4,507,498	4,513,475	4,511,998
4,000	8,009,998	8,017,966	8,015,998
5,000	12,512,498	12,522,469	12,519,998
6,000	18,014,998	18,026,963	18,023,998
7,000	24,517,498	24,531,457	24,527,998
8,000	32,019,998	32,035,954	32,031,998
9,000	40,522,498	40,540,463	40,535,998
10,000	50,024,998	50,044,945	50,039,998

Function Growth:



Conclusion:

This empirical analysis clearly demonstrates the rigid, non-adaptive nature of Selection Sort. Because the algorithm repeatedly scans the entire remaining unsorted section to find the absolute minimum element—before placing it in the correct position—the experimental data shows that its massive operation counts remain virtually identical across all input types.

Whether the input is perfectly sorted (Ascending), completely disorganized (Random), or completely backwards (Descending), the algorithm still forces the exact same number of heavy comparisons, proving its strict $O(n^2)$ time complexity in all cases. While it uses $O(1)$ extra memory space, this massive and unavoidable operation overhead makes Selection Sort highly inefficient and impractical for scaling datasets.